- It’s common to use a simple retry policy to try making a request multiple times before giving up and returning an error. When designing a policy, be sure to consider the effect of your policy; in some circumstances it may be better to fail quickly instead of retrying a request that is never going to succeed. Polly includes additional policies such as circuit-breakers to create more advanced approaches.
- By default, the transient error-handling middleware will handle connection errors, server errors that return a `5xx` error code, and `408` (timeout) errors. You can customize this if you want to handle additional error types but ensure that you retry only requests that are safe to do so.
- You can create a custom `HttpMessageHandler` to modify each request made through a named or typed client. Custom handlers are good for implementing cross-cutting concerns such as logging, metrics, and authentication.
- To create a custom `HttpMessageHandler`, derive from `DelegatingHandler` and override the `SendAsync()` method. Call `base.SendAsync()` to send the request to the next handler in the pipeline and finally to the `HttpClientHandler`, which makes the HTTP request.
- Register your custom handler in the DI container as either a transient or a singleton. Add it to a named or typed client using `AddHttpMessageHandler<T>()`. The order in which you register the handler in the `IHttpClientBuilder` is the order in which the handler will appear in the `HttpClient` handler pipeline.